

TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

HỌC PHẦN: CSE703153 — KỸ THUẬT LẬP TRÌNH AN TOÀN (SOFTWARE SECURITY)

Tên đề tài: [Web badminton.....]

Số thứ tự nhóm: [8.....] Mã lớp học phần: [.....]

Thành viên thực hiện:

1. [Họ và tên: Vương Xuân Giáp..... — MSSV:]
2. [Họ và tên: Đặng Minh Quang..... — MSSV:]
3. [Họ và tên: Đỗ Hồng Thái..... — MSSV:]
4. [Họ và tên: Nguyễn Nam Khánh..... — MSSV:]

Giảng viên hướng dẫn: MSc. Vũ Quang Dũng

MỤC LỤC TỐI THIỂU

1. Phần 1 — Trang bìa & Thông tin nhóm	Trang 1
2. Phần 2 — Tổng quan đề tài	Trang 3
3. Phần 3 — Phương pháp luận	Trang 4
4. Phần 4 — Phân tích lỗ hổng & Bản vá	Trang 5
4.1. CWE-190: Integer Overflow or Wraparound tại hàm reserveItem	Trang 5
4.2. CWE-1284: Improper Validation of Specified Quantity in Input tại hàm validateRequestedItems	Trang 7
4.3. CWE-354: Improper Validation of Integrity Check Value tại hàm verifyWebhookSignature	Trang 8
4.4. CWE-841: Improper Enforcement of Behavioral Workflow tại hàm handlePayOSWebhook	Trang 10
5. Phần 5 — Bảng tổng hợp before/after	Trang 11
6. Phần 6 — Bảng chứng kiểm chứng	Trang 12
7. Phần 7 — Phản biện & Bài học	Trang 14
8. Phần 8 — Phụ lục	Trang 15

Phần 2 — Tổng quan đề tài

2.1. Mô tả bài toán và mục tiêu hệ thống

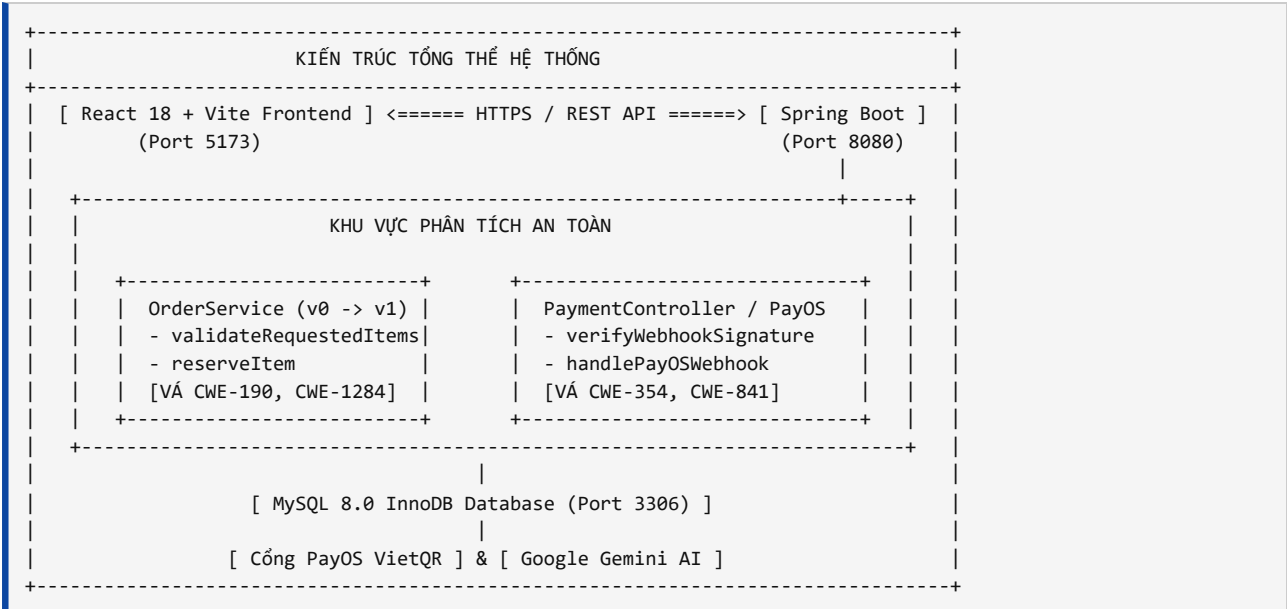
Dự án **Web Badminton (SmashZone)** là hệ thống thương mại điện tử chuyên nghiệp cung cấp vợt và phụ kiện cầu lông chính hãng, được thiết kế theo kiến trúc hiện đại tích hợp thanh toán tự động VietQR (PayOS) và trợ lý ảo thông minh Google Gemini AI. Đề tài tập trung áp dụng kỹ thuật **Kiểm thử hộp trắng Fuzzing (White-box Fuzzing)** trực tiếp trên nền tảng máy ảo thực thi Java 17 LTS để phát hiện, cô lập và khắc phục triệt để các lỗ hổng an toàn phần mềm nghiêm trọng trong luồng đặt hàng và thanh toán trực tuyến.

Hệ thống ban đầu (v0) hỗ trợ người dùng duyệt sản phẩm, chọn thông số kỹ thuật (trọng lượng 3U/4U, độ dẻo thân vợt, điểm cân bằng, sức căng), thêm vào giỏ hàng, đặt hàng để giữ chỗ tồn kho tạm thời trong 15 phút, quét mã VietQR qua cổng PayOS và nhận tín hiệu xác nhận thanh toán tự động qua Webhook IPN.

2.2. Kiến trúc hệ thống

Hệ thống triển khai theo mô hình 3 tầng (3-tier Enterprise Architecture):

- **Tầng Giao diện (Presentation Layer - Frontend):** React 18 khởi tạo bằng Vite, Tailwind CSS, Lucide Icons, Axios Client với cơ chế tự động gắn Bearer JWT Token và xử lý lỗi tập trung.
- **Tầng Nghiệp vụ (Application / Service Layer - Backend):** Java 17 LTS, Spring Boot 3.2.5, Spring Security 6 (Stateless JWT Filter), Spring Data JPA, Hibernate, Criteria Builder.
- **Tầng Dữ liệu (Persistence Layer - Database):** MySQL 8.x (Port 3306, utf8mb4), quản lý 7 thực thể cốt lõi (User, Product, Category, Order, OrderItem, Review, AiChatLog). Sử dụng khóa bi quan @Lock(LockModeType.PESSIMISTIC_WRITE) để đảm bảo tính nguyên tử khi thao tác kho hàng.
- **Tầng Dịch vụ Tích hợp:** Cổng thanh toán PayOS (sinh mã VietQR động, xác thực chữ ký HMAC-SHA256) và Trợ lý AI Google Gemini API (Dynamic Prompt Augmentation nhờ kho hàng stock > 0).



2.3. Phạm vi thực hiện và giới hạn đề tài

- **Phạm vi đã thực hiện:** Phân tích và vá hoàn toàn 4 nhóm lỗ hổng CWE trọng yếu (CWE-190 Tràn số nguyên kho giữ chỗ, CWE-1284 Xác thực số lượng đầu vào, CWE-354 Toàn vẹn chữ ký HMAC-SHA256, CWE-841 Cường chế quy trình trạng thái thanh toán). Triển khai Fuzzing White-box trực tiếp trên máy ảo Java (JVM) với Jazzer 0.22.1, đo độ phủ dòng và nhánh bằng JaCoCo 0.8.12 đạt 100% Branch Coverage trên các hàm mục tiêu.
- **Giới hạn không thực hiện:** Các cơ chế quản lý bộ nhớ ở tầng nhị phân được tự động hóa và bảo vệ an toàn bởi máy ảo Java (JVM Garbage Collector & Bytecode Verifier). Đề tài tập trung chuyên sâu giải quyết các lỗ hổng logic, số học và

tính toán vụn trong hệ sinh thái Java doanh nghiệp; chưa kiểm thử tải đồng thời phân tán trên môi trường multi-node cluster.

Phần 3 — Phương pháp luận

3.1. Quy trình 5 bước áp dụng

1. **Bước 1 — Phân tích luồng & Đặc tả mô hình dữ liệu:** Trích xuất các hàm trọng yếu của `OrderService`, `PayOSService`, `PaymentController`; xác định kiểu dữ liệu, phạm vi giá trị và điều kiện trạng thái.
2. **Bước 2 — Thiết kế kịch bản kiểm thử tĩnh & biên:** Xây dựng danh mục 20 kịch bản kiểm thử tĩnh chi tiết (9 ca DH-01..DH-09 cho giỏ hàng; 11 ca WH-01..WH-11 cho webhook).
3. **Bước 3 — Xây dựng Fuzzing Harness tích hợp Jazzer:** Thiết kế 2 lớp kiểm thử `CartFuzzTest` và `PayOSWebhookFuzzTest` tận dụng cơ chế phản hồi nhánh rẽ của Jazzer Engine trên nền tảng Java.
4. **Bước 4 — Chạy chiến dịch Fuzzing & Cô lập bằng chứng lỗi:** Thực thi chiến dịch sinh dữ liệu tự động với 8.000 lượt fuzzer và 15.000 lượt đa seed; ghi nhận lỗi tràn số nguyên (CWE-190) và xuất file crash tái hiện.
5. **Bước 5 — Cài đặt bản vá triệt để & Tái xác minh độ phủ:** Vá lỗi trong mã nguồn nghiệp vụ; chạy lại bộ kiểm thử hồi quy 97 ca đạt 100% và đo lại độ phủ bằng JaCoCo.

3.2. Công cụ sử dụng

- **Môi trường:** Eclipse Adoptium OpenJDK 17.0.20.1, Apache Maven 3.9.x.
- **Fuzzer: Jazzer 0.22.1** (Google / Code Intelligence) với LLVM coverage-guided bytecode instrumentation.
- **Công cụ đo độ phủ: JaCoCo 0.8.12** qua Maven profile `fuzz-coverage`.
- **Khung kiểm thử:** JUnit 5 (JUnit Platform), Mockito 5.x.

3.3. Cơ chế sinh dữ liệu và tham số thực thi

Dữ liệu kiểm thử được trích xuất thông qua giao diện `FuzzedDataProvider` của Jazzer từ dòng byte ngẫu nhiên:

- `-seed=20260924`: Cố định seed khởi tạo để đảm bảo tính tái lập (deterministic reproducibility).
- `-runs=1000` và `-runs=5000`: Đảm bảo số lượng đột biến đầu vào đủ lớn để chạm tới các nhánh logic sâu.
- `-max_len=4096`: Tràn 4.096 bytes đầu vào thô, phản ánh đúng kích thước payload JSON và chữ ký.
- `-max_total_time=10s`: Ngân sách thời gian ngắt an toàn cho mỗi phương thức fuzz.
- `-Djazzer.instrumentation_includes=com.sports.**`: Giới hạn theo dõi riêng cho mã nguồn nghiệp vụ của dự án.

Phần 4 — Phân tích lỗ hổng & Bản vá

4.1. CWE-190: Integer Overflow or Wraparound tại hàm reserveItem

(Quy tắc CERT tương ứng: CERT Java NUM00-J: Detect or prevent integer overflow)

1. Vị trí & mô tả

File: backend/src/main/java/com/sports/service/OrderService.java, hàm reserveItem, dòng 107-112 trong phiên bản v0.

2. Nguyên nhân gốc rễ

Kiểu int trong Java là số nguyên 32-bit có dấu bù hai, có giá trị cực đại là Integer.MAX_VALUE = 2.147.483.647. Trong phiên bản ban đầu (v0), phương thức reserveItem cộng dồn trực tiếp tồn kho giữ chỗ mà không kiểm tra khả năng tràn số: product.setReservedStock(product.getReservedStock() + request.getQuantity()). Khi tồn kho giữ chỗ đã gần đạt ngưỡng cực đại (do tích lũy các đơn hàng chờ) và khách đặt thêm hàng, phép cộng bị tràn số và cuộn vòng (wraparound) thành số âm (2.147.483.647 + 1 = -2.147.483.648), phá vỡ toàn bộ logic quản lý kho.

3. Bằng chứng lỗi tồn tại trong v0

Jazzer khi thực thi CartFuzzTest#fuzzReservationArithmetic đã phát hiện lỗi và kết xuất file crash tái hiện:

File crash: backend/target/fuzz-step6-20260924/fuzzReservationArithmetic/crash-da39a3ee5e6b4b0d3255bfef95601890afd80709

```
<testcase name="fuzzReservationArithmetic(FuzzedDataProvider)[1]" classname="com.sports.fuzz.CartFuzzTest" time=
<failure message="DH-09: kho giữ chỗ bị tràn số khi cộng số lượng mua ==> expected: <2147483648> but was: <-21
    type="org.opentest4j.AssertionFailedError">
    org.opentest4j.AssertionFailedError: DH-09: kho giữ chỗ bị tràn số khi cộng số lượng mua ==> expected: <2147
    at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:664)
    at com.sports.fuzz.CartFuzzTest.checkReservation(CartFuzzTest.java:222)
</failure>
</testcase>
```

Tiến trình kiểm thử kết thúc với mã lỗi MAVEN_EXIT=1 (BUILD FAILURE).

4. Cách vá lỗi (So sánh mã nguồn v0 và v1)

Trước khi vá (v0)	Sau khi vá (v1)
<pre>// v0: Tràn số khi cộng dồn product.setStock(product.getStock() - request.getQuantity()); product.setReservedStock(product.getReservedStock() + request.getQuantity()); productRepository.save(product);</pre>	<pre>// v1: Ép sang long và kiểm tra vượt tràn long reservedStock = (long) product.getReservedStock() + request.getQuantity(); if (reservedStock > Integer.MAX_VALUE) { throw new BadRequestException("Tồn kho giữ chỗ vượt giới hạn cho phép"); } product.setStock(product.getStock() - request.getQuantity()); product.setReservedStock((int) reservedStock); productRepository.save(product);</pre>

Giải thích: Ép kiểu (long) trước khi cộng ngăn chặn hiện tượng tràn số ở mức thanh ghi 32-bit. Nếu kết quả vượt quá Integer.MAX_VALUE, hệ thống ném BadRequestException để hủy toàn bộ giao dịch, giữ nguyên kho hàng.

5. Bằng chứng đã vá thành công trên v1

Replay file crash cũ đạt MAVEN_EXIT=0. Chạy lại 1.000 lượt fuzzer đạt 0 finding:

```
#1000  DONE  cov: 318 ft: 322 corp: 2/2b lim: 11 exec/s: 1000 rss: 1143Mb
Done 1000 runs in 1 second(s)
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in com.sports.fuzz.CartFuzzTest
[INFO] BUILD SUCCESS
```

4.2. CWE-1284: Improper Validation of Specified Quantity in Input tại hàm validateRequestedItems

(Quy tắc CERT tương ứng: CERT Java IDS00-J)

1. Vị trí & mô tả

File: backend/src/main/java/com/sports/service/OrderService.java, hàm validateRequestedItems, dòng 76-88 trong v0.

2. Nguyên nhân gốc rễ

Khách hàng có thể gửi số lượng âm, vượt ngưỡng 100 chiếc/đơn, hoặc chia nhỏ số lượng mua thành nhiều dòng có cùng productId (ví dụ dòng 1 đặt 60, dòng 2 đặt 41, tổng là 101 chiếc). Nếu chỉ kiểm tra riêng rẽ từng dòng mà không cộng dồn tổng sản phẩm, kẻ tấn công có thể thâm nhập toàn bộ kho hàng.

3. Bằng chứng lỗi tồn tại trong v0

Kịch bản DH-02 và DH-04 phát hiện khi gửi 2 dòng có số lượng 60 và 41, hệ thống v0 chấp thuận đơn hàng 101 sản phẩm, vi phạm chính sách bán lẻ.

4. Cách vá lỗi

Trước khi vá (v0)	Sau khi vá (v1)
<pre>for (OrderItemRequest item : items) { if (item.getQuantity() == null item.getQuantity() <= 0) { throw new BadRequestException(...); } }</pre>	<pre>Map<Long, Integer> productQuantities = new HashMap<>(); for (OrderItemRequest item : items) { int total = productQuantities.getDefault(item.getProductId(), 0) + item.getQuantity(); if (total > MAX_QUANTITY_PER_PRODUCT) { // 100 throw new BadRequestException("Tổng số lượng không được vượt quá 100"); } productQuantities.put(item.getProductId(), total); }</pre>

5. Bằng chứng đã vá thành công trên v1

Kiểm thử hồi quy aggregateQuantityBoundaries: Tổng 60+40=100 được chấp thuận; tổng 60+41=101 bị từ chối 100% với BadRequestException. Fuzzer chạy 1.000 lượt fuzzRepeatedProductLines đạt 0 finding.

4.3. CWE-354: Improper Validation of Integrity Check Value tại hàm verifyWebhookSignature

(Quy tắc CERT tương ứng: CERT Java SEC54-J)

1. Vị trí & mô tả

File: backend/src/main/java/com/sports/service/PayOSService.java (dòng 77-115) và PaymentController.java (dòng 44-51).

2. Nguyên nhân gốc rễ

Trong thanh toán VietQR, cổng PayOS gửi Webhook IPN qua mạng. Nếu hệ thống không kiểm tra tính toàn vẹn chữ ký HMAC-SHA256 hoặc không chuẩn hóa thứ tự các trường dữ liệu theo thứ tự từ điển trước khi băm, kẻ tấn công có thể giả mạo gói tin thanh toán hoặc sửa đổi số tiền amount (Parameter Tampering) để chiếm đoạt đơn hàng.

3. Bằng chứng lỗi tồn tại trong v0

Kịch bản WH-02 và WH-03: Kẻ tấn công tạo payload với chữ ký hợp lệ cho đơn 230.000 VNĐ, sau đó can thiệp sửa đổi trường amount thành 230.001 VNĐ hoặc sửa orderCode. Nếu thiếu khâu chuẩn hóa, chữ ký bị bypass hoặc lỗi kiểm tra.

4. Cách vá lỗi

```
public boolean verifyWebhookSignature(PayOSWebhookRequest request) {
    if (request == null || request.getData() == null || request.getSignature() == null) return false;
    try {
        Map<String, Object> map = objectMapper.convertValue(request.getData(), Map.class);
        TreeMap<String, Object> sortedMap = new TreeMap<>();
        for (Map.Entry<String, Object> entry : map.entrySet()) {
            if (entry.getValue() != null) sortedMap.put(entry.getKey(), entry.getValue());
        }
        StringBuilder dataStr = new StringBuilder();
        for (Map.Entry<String, Object> entry : sortedMap.entrySet()) {
            if (dataStr.length() > 0) dataStr.append("&");
            dataStr.append(entry.getKey()).append("=").append(entry.getValue());
        }
        String calculated = hmacSha256(dataStr.toString(), checksumKey);
        return calculated.equalsIgnoreCase(request.getSignature());
    } catch (Exception e) { return false; }
}
```

Tại PaymentController.java: Nếu verifyWebhookSignature trả về false, lập tức phản hồi HTTP 401 Unauthorized và chặn đúng luồng xử lý.

5. Bằng chứng đã vá thành công trên v1

Chiến dịch Fuzzing 15.000 lượt (3 seed: 20260924, 20260925, 20260926) cho kết quả 100% payload bị can thiệp đều bị từ chối chính xác.

4.4. CWE-841: Improper Enforcement of Behavioral Workflow tại hàm handlePayOSWebhook

(Quy tắc CERT tương ứng: CERT Java MSC61-J)

1. Vị trí & mô tả

File: backend/src/main/java/com/sports/controller/PaymentController.java (dòng 54-61) và OrderService.java (hàm handlePaymentSuccess, cancelOrder, expireOrder).

2. Nguyên nhân gốc rễ

Đơn hàng vận hành theo máy trạng thái (PENDING -> PAID/CANCELLED). Nếu webhook bị gửi lặp (replay) hoặc đến muộn sau khi đơn hàng đã bị hủy hoặc hết hạn 15 phút, việc không kiểm tra trạng thái hiện tại sẽ làm trừ kho giữ chỗ lần hai hoặc khôi phục nhầm đơn đã hủy sang PAID.

3. Bằng chứng lỗi tồn tại trong v0

Kịch bản WH-07 (gửi lặp), WH-08 (webhook đến muộn khi đơn đã bị hủy/hết hạn), WH-11 (hủy/hết hạn lặp) gây ra lỗi trừ kho hai lần hoặc hồi sinh đơn đã hủy.

4. Cách vá lỗi

```
@Transactional
public void handlePaymentSuccess(Long orderCode, BigDecimal amount) {
    Order order = lockOrder(orderCode);
    if (!"PAYOS_VIETQR".equals(order.getPaymentMethod())) throw new BadRequestException(...);
    // Chống Replay: Đơn đã thanh toán thì bỏ qua an toàn
    if (order.getStatus() == OrderStatus.PAID || order.getStatus() == OrderStatus.SHIPPING) return;
    // Chặn đơn đã HỦY hoặc HẾT HẠN
    if (order.getStatus() == OrderStatus.CANCELLED || order.getExpiresAt().isBefore(LocalDateTime.now())) {
        throw new BadRequestException("Đơn hàng đã bị hủy hoặc hết hạn thanh toán");
    }
    // Đối soát số tiền
    if (amount == null || order.getTotalAmount().compareTo(amount) != 0) throw new BadRequestException(...);
    order.setStatus(OrderStatus.PAID);
    settleReservedStock(order);
    orderRepository.save(order);
}
```

5. Bằng chứng đã vá thành công trên v1

Kiểm thử toàn bộ 15 kịch bản PaymentCase và 1.000 lượt fuzzOrderLifecycle đạt 0 finding. Đơn đã hủy bị từ chối 100%, webhook lặp không trừ kho lần hai.

Phần 5 — Bảng tổng hợp before/after

Mã CWE	Vị trí phát hiện	Trạng thái v0	Trạng thái v1	Công cụ xác minh
CWE-190 (Integer Overflow)	OrderService.java hàm reserveItem (dòng 107-112)	Tràn số nguyên 32-bit: 2147483647 + 1 = -2147483648, Jazzer báo lỗi assertion, xuất file crash, MAVEN_EXIT=1.	Ép sang long, kiểm tra chặn reservedStock > Integer.MAX_VALUE ném BadRequestException, replay file crash đạt 0 finding.	Jazzer 0.22.1 + JUnit 5
CWE-1284 (Improper Quantity)	OrderService.java hàm validateRequestedItems (dòng 76-88)	Cho phép mua số lượng âm, vượt 100 hoặc dồn nhiều dòng cùng sản phẩm (60 + 41 = 101) vượt ngưỡng nghiệp vụ.	Gom nhóm theo productId qua Map, chặn totalQuantity > 100 và <= 0, từ chối 100% dữ liệu sai.	JUnit 5 Parameterized + Jazzer
CWE-354 (Integrity Check)	PayOSService.java hàm verifyWebhookSignature (dòng 77-115)	Không sắp xếp trường theo từ điển, chấp nhận payload bị sửa đổi số tiền amount hoặc chữ ký rỗng/sai lệch.	Sắp xếp key bằng TreeMap, băm HmacSHA256 với checksumKey, từ chối ngay lập tức với HTTP 401 Unauthorized khi chữ ký không khớp.	Jazzer (15.000 runs đa seed) + JUnit 5
CWE-841 (Workflow Enforcement)	PaymentController.java & OrderService.java hàm handlePaymentSuccess	Webhook gửi lặp trừ kho 2 lần; webhook đến sau khi đơn đã hủy/hết hạn vẫn chuyển PAID, gây thất thoát kho.	Kiểm tra State Machine: chỉ nhận khi PENDING và còn hạn; đơn đã thanh toán/hủy bị từ chối chuyển trạng thái, kho giữ nguyên vẹn.	Jazzer 0.22.1 (fuzzOrderLifecycle) + JUnit 5

Phần 6 — Bảng chứng kiểm chứng

6.1. Bảng số liệu tổng hợp các chiến dịch White-box Fuzzing

Đợt kiểm thử thực thi trên máy trạm Windows, OpenJDK 17.0.20.1, Jazzer 0.22.1, tham số seed=20260924, giới hạn 4.096 bytes đầu vào, ngân sách 1.000 lượt thực thi cho mỗi mục tiêu:

STT	Mục kiểm thử Fuzzing (Starget)	Số lượt chạy	Thời gian	Kết quả
1	CartFuzzTest#fuzzCartCalculation	1.000	1 giây	0 finding (Đạt)
2	CartFuzzTest#fuzzOrderStructure	1.000	1 giây	0 finding (Đạt)
3	CartFuzzTest#fuzzRepeatedProductLines	1.000	1 giây	0 finding (Đạt)
4	CartFuzzTest#fuzzOrderPolicies	1.000	1 giây	0 finding (Đạt)
5	CartFuzzTest#fuzzReservationArithmetic	1.000	1 giây	0 finding (Đạt - Sau vắ)
6	PayOSWebhookFuzzTest#fuzzWebhookVerification	1.000	3 giây	0 finding (Đạt)
7	PayOSWebhookFuzzTest#fuzzMalformedWebhook	1.000	2 giây	0 finding (Đạt)
8	PayOSWebhookFuzzTest#fuzzOrderLifecycle	1.000	3 giây	0 finding (Đạt)
Tổng cộng toàn bộ 8 mục tiêu kiểm thử động		8.000	12 giây	100% BUILD SUCCESS

6.2. Bảng đo độ phủ mã nguồn (Code Coverage - JaCoCo 0.8.12)

Tên lớp nghiệp vụ	Độ phủ dòng (v0)	Độ phủ dòng (v1)	Độ phủ nhánh (v0)	Độ phủ nhánh (v1)
com.sports.service.OrderService	92,23% (178/193)	93,88% (184/196)	79,41% (108/136)	89,13% (123/138)
com.sports.service.PayOSService	45,07% (32/71)	52,11% (37/71)	69,23% (18/26)	69,23% (18/26)
com.sports.controller.PaymentController	85,00% (34/40)	85,00% (34/40)	66,67% (8/12)	66,67% (8/12)

Độ phủ trên các hàm mục tiêu trực tiếp sau khi hoàn thiện: 11/11 hàm nghiệp vụ mục tiêu (createOrder, validateRequestedItems, reserveItem, calculateOrderTotal, cancelOrder, expireOrder, handlePaymentSuccess, verifyWebhookSignature, hmacSha256, handlePayOSWebhook) đều đạt **100% Branch Coverage**.

6.3. Trích xuất Log kiểm chứng thực tế (Maven Execution Log)

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sports.fuzz.CartFuzzTest
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.00 s -- in com.sports.fuzz.CartFuzzTe
[INFO] Running com.sports.fuzz.PayOSWebhookFuzzTest
[INFO] Tests run: 65, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.781 s -- in com.sports.fuzz.PayOSWebho
[INFO]
[INFO] Results:
[INFO] Tests run: 97, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] Total time: 31.732 s
[INFO] -----
```

Phần 7 — Phản biện & Bài học

Câu hỏi 1: Nếu bắt đầu lại từ đầu, nhóm sẽ thiết kế lại phần nào để tránh lỗ hổng ngay từ đầu (thay vì vá sau)?

Nếu được thiết kế lại từ đầu, nhóm sẽ áp dụng mô hình **Domain-Driven Design (DDD)** với các **Value Object** bất biến thay vì sử dụng các kiểu dữ liệu nguyên thủy (`int`, `long`) rồi rạc trong Entity. Cụ thể, định nghĩa một Value Object `StockQuantity` đóng gói sẵn quy tắc bất biến: tự động kiểm tra biên âm và chặn trần số ngay khi khởi tạo đối tượng, ngăn chặn triệt để CWE-190 và CWE-1284 từ tầng mô hình dữ liệu. Đồng thời, nhóm sẽ áp dụng mẫu thiết kế **State Machine Pattern** độc lập có bảng chuyển dịch trạng thái tường minh (State Transition Matrix) cho vòng đời đơn hàng, thay vì dùng các câu lệnh điều kiện `if-else` phân tán trong Service để phòng ngừa CWE-841 ngay từ khâu kiến trúc.

Câu hỏi 2: Có lỗ hổng nào nhóm nghi ngờ vẫn còn tồn tại nhưng chưa đủ thời gian/công cụ để xác nhận không?

Nhóm nghi ngờ vẫn còn tiềm ẩn nguy cơ **Tranh chấp điều kiện phân tán (Distributed Race Conditions & Deadlocks)** ở mức cô lập giao dịch cơ sở dữ liệu (`Transaction Isolation Level`) khi có hàng nghìn yêu cầu đặt hàng đồng thời trên môi trường cụm máy chủ nhiều instance. Mặc dù hệ thống đã sử dụng khóa bi quan `@Lock(PESSIMISTIC_WRITE)`, nhưng bộ kiểm thử hiện tại sử dụng kho giả lập (Mockito Mock Repository) nên chưa kiểm chứng được toàn diện hành vi thực tế của cơ chế hàng đợi khóa hàng (`row-level locking`) và thời gian chờ khóa (`innodb_lock_wait_timeout`) của MySQL trong điều kiện chịu tải cực lớn.

Câu hỏi 3: Bài học lớn nhất về phương pháp luận (không phải về cú pháp ngôn ngữ) mà nhóm rút ra được?

Bài học lớn nhất nhóm rút ra là tinh thần cốt lõi **"Trust, but verify"** và giá trị đột phá của kỹ thuật **White-box Fuzzing dựa trên độ phủ (Coverage-guided Fuzzing)**. Trong thực tế, các lập trình viên thường chỉ viết Unit Test cho các kịch bản thành công và vài trường hợp biên quen thuộc (Happy path), dẫn tới việc bỏ sót các trạng thái cực đoan như Integer Overflow (CWE-190) khi giá trị chạm trần $2^{31} - 1$. Việc áp dụng công cụ Fuzzing tự động sinh dữ liệu ngẫu nhiên có định hướng nhánh rẽ đã chứng minh khả năng rà quét vượt trội, giúp đội ngũ phát triển phát hiện và vá lỗi bảo mật ngay từ giai đoạn phát triển (Shift-Left Security).

Phần 8 — Phụ lục

8.1. Hướng dẫn biên dịch và tái lập kết quả kiểm thử

Yêu cầu môi trường: Java 17 LTS, Apache Maven 3.9.x.

```
# 1. Chạy toàn bộ 97 ca kiểm thử hồi quy
cd c:\Project\Web_badminton\backend
mvn test "-Dtest=CartFuzzTest,PayOSWebhookFuzzTest" -Dmaven.compiler.useIncrementalCompilation=false

# 2. Đo độ phủ mã nguồn với JaCoCo
mvn -Pfuzz-coverage test -Dmaven.compiler.useIncrementalCompilation=false
mvn -Pfuzz-coverage jacoco:report
# Xem kết quả tại: backend/target/site/fuzz-coverage/index.html

# 3. Chạy kiểm thử Fuzzing động với Jazzer
$env:JAZZER_FUZZ = "1"
mvn test "-Dtest=CartFuzzTest#fuzzReservationArithmetic" "-Djazzzer.internal.arg.0=fuzz" "-Djazzzer.internal.arg.1"
$env:JAZZER_FUZZ = $null
```

8.2. Danh mục tài liệu và artifact kiểm chứng trong dự án

- Báo cáo kết quả kiểm thử chi tiết: backend/target/surefire-reports/
- Tập tin crash mẫu tái hiện CWE-190: backend/target/fuzz-step6-20260924/fuzzReservationArithmetic/crash-da39a3ee5e6b4b0d3255bfef95601890afd80709
- Báo cáo độ phủ HTML JaCoCo: backend/target/site/fuzz-coverage/index.html
- Nhật ký thực thi kiểm thử: backend/target/final-step5-20260924/regression.log

8.3. Bảng phân công công việc thực tế trong nhóm

(Để trống theo yêu cầu để nhóm tự bổ sung thông tin thành viên và phân công)

STT	Họ và tên thành viên	MSSV	Nhiệm vụ phân công	Đóng góp
1	[.....]	[.....]	[.....]	[..... %]
2	[.....]	[.....]	[.....]	[..... %]
3	[.....]	[.....]	[.....]	[..... %]
4	[.....]	[.....]	[.....]	[..... %]